

Jour 10 : Fermeture de connexion et Tests finaux

1. Handshake de Fermeture (4-way) :

- Dans tcp.c, j'ai ajouté la détection du drapeau FIN.
- Lorsqu'un client (comme curl) demande à fermer la connexion, la pile envoie un ACK, passe en état LAST_ACK, envoie son propre FIN, puis libère les ressources (CLOSED) une fois le dernier acquittement reçu.

2. Gestion de l'incrément SEQ/ACK : J'ai fait que le FIN (tout comme le SYN) utilise bien un numéro de séquence, garantissant que l'arithmétique TCP reste cohérente lors de la fermeture.

3. Nettoyage des TCB : Les slots de connexion sont désormais correctement libérés, permettant à de nouvelles connexions de réutiliser les ressources sans blocage.

Focus Sécurité : L'Attaque RST (Reset Injection)

C'est une attaque simple.

- **Le Concept** : Un attaquant (ou un pare-feu) peut forger un paquet TCP avec le drapeau RST activé et l'envoyer à votre serveur en usurpant l'identité du client.
- **L'Effet** : Dès que votre pile reçoit ce RST, elle coupe INSTANTANÉMENT la connexion et libère les ressources. C'est le moyen le plus rapide de saboter une communication SSH ou un transfert de fichier.
- **Vulnérabilité** : Dans notre pile simplifiée, un RST est accepté s'il arrive sur une connexion active. Dans une pile réelle, il y a des vérifications strictes sur la fenêtre de calcul du numéro de séquence pour éviter que n'importe qui puisse "tuer" une session de l'extérieur.

Après avoir lancé le server, on doit faire une requête

```
3. 192.168.56.106 x 4. /home/mobaxterm x 5. 192.168.56.106 x
gcc -Wall -Wextra -I. -c socket.c -o socket.o
gcc -Wall -Wextra -I. -c tun.c -o tun.o
gcc main.o arp.o ip.o icmp.o tcp.o tcp_utils.o socket.o tun.o -o arp_stack
Interface tap0 créée. Configurez: sudo ip addr add 10.0.0.1/24 dev tap0 && sudo
ip link set tap0 up
SOCKET: Système initialisé.
SOCKET: Création socket ID 1
SOCKET: Socket 1 lié au port 80 (LISTEN)
APP: Serveur Web simulé en écoute sur le port 80 via API Socket.

--- Cache ARP Interne ---
[0] 10.0.0.1 -> 26:be:97:05:80:df
-----
ARP Reply envoyé à 10.0.0.2

--- Paquet IP reçu ---
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 60
TCP: Requête de connexion SYN de 10.0.0.1:49610
TCP STATE: LISTEN -> SYN_RECEIVED
TCP: SYN-ACK envoyé

--- Paquet IP reçu ---
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 40
TCP STATE: SYN_RECEIVED -> ESTABLISHED

--- Paquet IP reçu ---
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 112
TCP DATA: [72 octets] reçus. Transfert vers Socket API.
TCP: ACK envoyé (SEQ: 9384, ACK attendu: 3968171608)

--- APPLICATION USER-SPACE ---
Reçu 72 octets sur le port 80 :
GET / HTTP/1.1
Host: 10.0.0.2
User-Agent: curl/8.17.0
Accept: */*

-----

TCP TIMEOUT: Retransmission pour TCB 0 (Essai 1/5, RT0: 3s)
```

1. Le Handshake et la Réception

- **SYN -> SYN-ACK -> ESTABLISHED** : Le handshake s'est déroulée parfaitement sur le port 49610.
- **C'est OK**

2. Pourquoi ces Timeouts et Erreurs ?

Après avoir reçu les données, on voit des TCP TIMEOUT et des TCP ERROR: SEQ invalide. Voici pourquoi :

- **L'Application est muette** : le serveur a reçu la question de curl (GET /), mais il n'a pas renvoyé de réponse (car nous n'avons pas codé la partie xsend pour répondre "HTTP/1.1 200 OK").
- **Curl attend** : Comme curl n'a pas reçu de réponse, il reste en attente.
- **Retransmission du Client** : curl (ou le noyau Linux hôte) finit par croire que son paquet de données a été perdu (puisque'il n'a pas "vu" passer votre ACK ou que la réponse n'arrive pas). Il renvoie donc son paquet.
- **Conflit de Séquence** : la stack reçoit à nouveau les données. Elle compare le numéro de séquence reçu (3968171607) avec celui qu'elle attend désormais (3968171608). Comme ils diffèrent de 1, elle affiche SEQ invalide et renvoie un ACK pour dire "Non, je t'attends à 3968171608".
 - **C'est le comportement standard de TCP** : On appelle cela un **Duplicate ACK**. la pile se défend correctement ! c'est toujours bien.

3. Pourquoi vous ne voyez pas le "FIN" ?

Dans vos logs, la connexion n'est pas encore fermée. Pourquoi ?

- curl attend désespérément une réponse. Il ne fermera la connexion que s'il reçoit une réponse ou s'il atteint son propre timeout (souvent très long, genre 2 minutes).

voici ce qui devrait se passer normalement (on lance le curl puis on le coupe au bout d'environ 30 secondes pour pas qu'il attende trop longtemps et génère les mêmes erreurs qu'avant)

```
-----  
--- Paquet IP reçu ---  
Source: 10.0.0.1  
Destination: 10.0.0.2  
Protocol: 6  
Total Length: 40  
TCP FIN: Requête de fermeture reçue de 10.0.0.1:59282  
TCP: ACK envoyé (SEQ: 887, ACK attendu: 711731924)  
TCP STATE: ESTABLISHED -> LAST_ACK  
TCP: FIN-ACK envoyé  
  
--- Paquet IP reçu ---  
Source: 10.0.0.1  
Destination: 10.0.0.2  
Protocol: 6  
Total Length: 40  
TCP STATE: LAST_ACK -> CLOSED. Ressources libérées.
```